OMNITICKET ACADEMIC

An Event Ticketing and QR code Entry System for University Societies

FINAL YEAR PROJECT REPORT DRAFT 1

STUDENT NAME: STEPHEN KERRY

STUDENT NUMBER:20106900

PROGRAMME: SOFTWARE SYSTEMS DEVELOPMENT

SUPERVISOR: DIARMUID O CONNOR

DATE: 03/11/2026

Table of Contents

1 **Introduction** 2

1.1 Project Background 2

1.2 Problem Statement 2

1.3 Aim and Objectives of this Project 2

1.4 Scope 3

1.5 Proposed Solution 3

2 **Application Architecture** 4

2.1 Overview 4

2.2 Architecture Diagram 4

2.3 Design Decisions 5

2.4 Components 5

2.5 Main Flows 6

2.5.1 Registration and email verification 6

2.5.2 Ticket purchase 6

2.5.3 Door scanning 6

2.6 Concurrency Design 7

2.7 Redis and Background Workers 7

1 Introduction

1.1 Project Background

University societies are a vital part of student life. Throughout the school year, University societies can organise different things like Society Balls, Day Trips, and more. Many of these events sell tickets in limited numbers. Demand for popular events can get very high, and everything tends to happen all at once: When tickets are released, a lot of students tend to buy them within a short window of time. The society members who are selling the tickets are volunteers and in most cases, these sales are all done by hand.

In my personal experience, I remember my time buying a ticket for a science and computing ball and I can confirm it was a difficult experience because of the sheer amount of people trying to buy tickets the day they were released.

1.2 Problem Statement

Omniticket Academic aims to address two related problems.

The first is Ticket distribution. During busy peak periods, I can imagine that the sellers are often overwhelmed when they have to try and hand out tickets in person as physical distribution can be slow and it is difficult to keep an accurate count for how many tickets are sold and how many tickets remain. For a volunteer committee, this can be a significant burden.

The second is resale and verification. In some cases, students buy several tickets and sell them onto people who are not even students, so tickets who are originally intended only for society members do not even reach them. At the moment societies do not have a solid way to verify that the ticket buyer is a genuine student.

A system like OmniTicket that is built specifically for University societies can remove the physical step and restrict purchases to verified students.

1.3 Aim and Objectives of this Project

The primary aim of this project is to ultimately design and build a web-based event ticketing and entry system for university societies that eliminates physical ticket distribution and helps to verify ticket purchases to verified setu students only, never sells more tickets than actually exist and speeds up entry at the door.

The objectives for this project are as follows:

1. **Authentication :** Students/ Users will register up to Omniticket with an @setu.ie email address and they will prove that they own it by following a confirmation link sent to that specific email address.
2. **Ticket Allocation:** Background system that will handle a high volume of simultaneous ticket purchase requests without double-booking.
3. **QR Code Entry :** Each ticket will carry a secure, signed qr code. A browser based scanner will allow door staff validate tickets on entry, and each ticket will allow for one person only
4. **Analytics Dashboard:** Society members will create and manage events and be able to view live ticket sales and export reports if needed.

1.4 Scope

This report will aim to cover the first stage of the project. The work in this semester is the investigation and justification of the technology stack, the design of the database, the application architecture, the backend workers and Redis Components and the user interface wireframes. The purpose of this stage is to establish a solid foundation before the implementation begins next semester.

Implementation and testing of the full system will take place next semester.

1.5 Proposed Solution

OmniTicket Academic is a web application with 3 types of users: Students, Society Admins and door staff. The students will be able to register, verify their student address, look at different events and buy tickets and they will receive qr codes with the tickets. Society Admins will be able to create events and view sales and look at a financial dashboard. Door staff will scan tickets at the entrance.

The technology stack will be as follows: Django will be used, a python web framework which will provide the logic behind the app, user accounts and some security features. MySQL will be the database. It will store users, events and tickets and its transactions and row level locking will be used to stop the same ticket being sold twice. Redis and celery will be for background work, Celery will handle slow background tasks like sending emails and generating qr codes and Redis will be the broker and the queue between the web app and the workers and it can also hold tickets temporarily during a checkout. The front end will use html, CSS and JavaScript and will use html5-qrcode library for scanning in the browser.

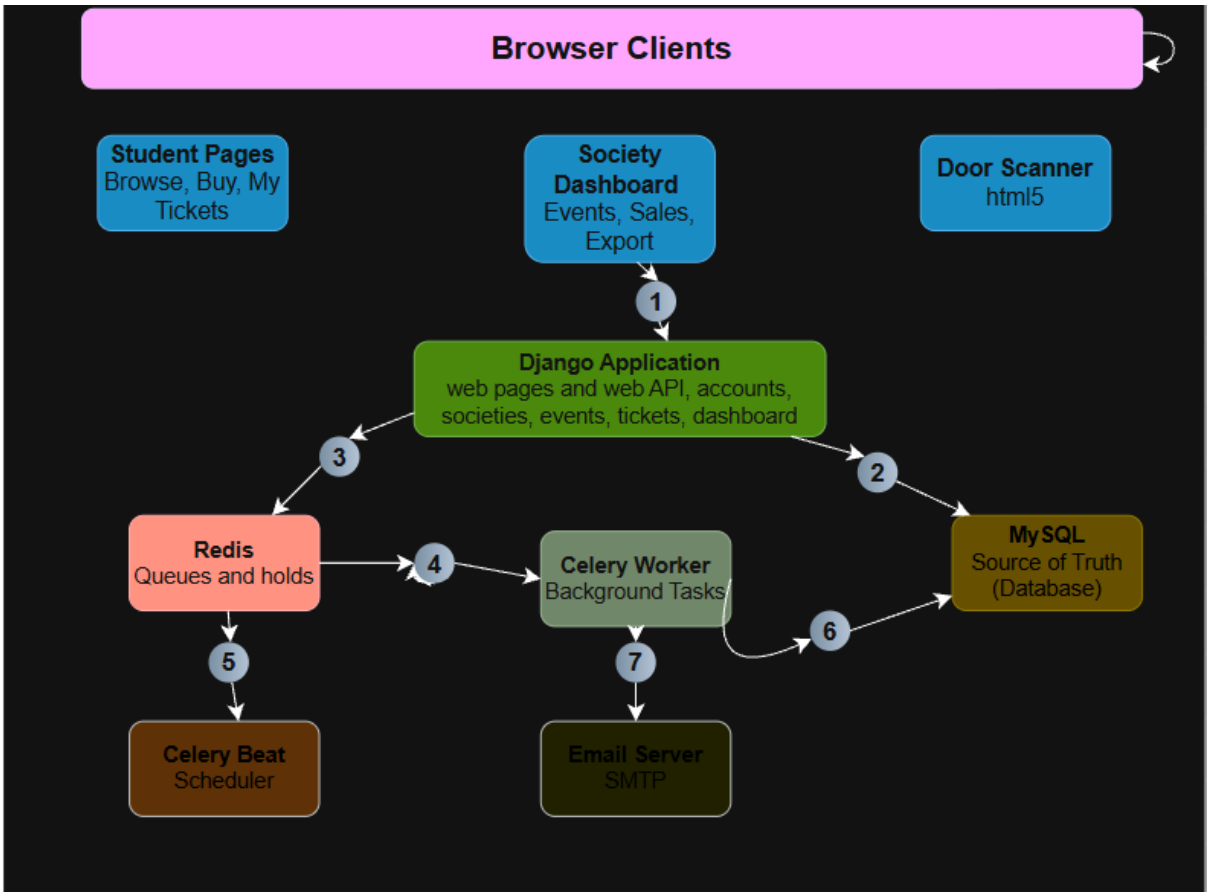
2 Application Architecture

2.1 Overview

Omniticket Academic is a web application backed by a relational database and a task queue. Browser client present the student pages, the society dashboard and the door scanner. The Django app will be divided into accounts, societies, events tickets and the dashboard, it serves the pages and a web API that will enforce rules. MySQL will act as the database that stores everything and it must be correct. Redis and Celery carry out slow and scheduled work away from the current web request. An email server must also be in here for sending the emails and optionally maybe a payment provider.

2.2 Architecture Diagram

Figure 2.1



1 Browsers send HTTPS requests to Django
2 Django reads and writes MySQL
3 Django queues tasks, sets ticket holds
4 Worker takes tasks off the queue
5 Beat adds scheduled tasks to the queue
6 Worker updates records in MySQL
7 Worker sends email via the SMTP server

2.3 Design Decisions

The decisions shape the design.

- **MySQL is source of truth.** The users, events and tickets and stock are protected by transactions in MySQL. ‘
- **Slow Work stays off the request path.** Emails, Qr codes, expired holds and report exports are handled by celery workers, so requests return quicker during a ticket rush.
- **The server enforces every rule.** Permissions, Ticket limits and QR validation are checked on the server because browsers are not truth.

2.4 Components

Table 2.1 lists the components and what each is responsible for

Components	Responsible	Technology
Browser Clients	Student Pages (browse, buy, show QR Codes), society dashboard (events, sales, export) and door scanner	HTML, CSS, Javascript, html5,
Django Application	Authentication, role checks, event and ticket logic, the Web API, and signing and checking of QR token.	Django(Python)
MySQL	Database Permanent Storage. Transactions and Row level locking prevent double booking.	MYSQL(INNODB)
Redis	Task Queue, and short lived ticket holds that expire automatically.	Redis
Celery Worker and Beat	The workers run background. Beat starts schedule, such as releasing expired holds	Celery
Email Server	Delivers verification and ticket email	SMTP

2.5 Main Flows

2.5.1 Registration and email verification

1. The student submits a name, an @setu.ie email address and a password. Django checks the domain and creates an unverified account with a hashed password.
2. Django creates a signed, expiring token and, once the account is saved, queues a task to send the verification emails.
3. A celery worker takes the task off the Redis queue and sends the email containing the link.
4. The student clicks the link, Django checks the token and marks the email as verified. Only verified accounts can buy tickets.

2.5.2 Ticket purchase

1. The student requests a ticket. Django checks they are logged in and verified.
2. In one MySQL Transaction, Django locks the event row with with select_for_update(), checks that tickets remain and that the student is under the per-student limit, reduces the remaining count and creates the ticket with a signed qr code.
3. If paymemt is required, the ticket is held and Django sets a redis key that expires after 5 minutes.
4. After the commit, Django queues the confirmation email and the QR code tasks and replies immediately.

A ticket is therefore held whyile the payment is pending, valid once bought, used once scanned and cancelled if its hold expires or it is refunded .

2.5.3 Door scanning

1. Door staff log in, and Django confirms their door staff or admin role for the society. The scanner page reads the QR code with the camera and sends the token to Django over HTTPS.
2. Django checks the token's signature.
3. In a transaction, Django locks the ticket's row. A valid ticket is marked used, with the time and staff member recorded, and accepted. A used ticket is reported as already used, and a held or cancelled ticket as invalid.
4. Django logs the attempt and will return the result, as a green, amber or red banner with the ticket holder's name.

Because the status change happens inside a lock, two simultaneous scans of the same code cannot both be accepted.

2.6 Concurrency Design

The main technical risk is a rush of purchases. If two requests both read "one ticket left" before either saves a change, a ticket is sold twice. This design guards against this in layers.

- **Row locking.** The purchase runs inside transaction.atomic() and locks the event row with select_for_update(). The second request waits, then sees zero and is refused. [Ref: Django documentation on select_for_update: https://docs.djangoproject.com/en/6.1/ref/databases/ .]
- **The right database.** MySQL's InnoDB engine supports row locking.
- **A database safety net.** A CHECK constraint stops the remaining count going below zero.
- **Temporary holds.** A Redis key created with SET … NX EX 300 reserves a ticket for five minutes, and only one request can create it. A scheduled task then releases expired holds. [Ref: Redis documentation on SET: https://redis.io/docs/latest/commands/set/]

Transactions are kept short, rows are locked in a consistent order, and slow work such as email is queued after the commit rather than done inside a lock, which reduces the risk of deadlocks.

2.7 Redis and Background Workers

Redis has two roles. It is the Celery broker, holding the queue between Django and the worker, and it stores the temporary ticket holds. Redis keeps data in memory, so queued messages can be lost in a crash [Ref: Redis documentation on persistence: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/]. This is acceptable because MySQL is the source of truth: a lost email task can be sent again, and the release task treats a hold as expired if its Redis key is missing, so stock is never left stuck. Table 2.2 lists the background tasks.

Table 2.2: Background tasks

Task	Triggered by	Purpose	Schedule
send_verification_email	Registration	Emails the confirmation link.	On demand
send_ticket_confirmation	Purchase	Emails the ticket to the buyer.	On demand
generate_qr_image	Purchase	Creates the QR image for a ticket.	On demand
release_expired_holds	Celery beat	Cancels expired holds and restores stock.	Every minute
export_sales_report	Society admin	Builds a CSV of sales for an event.	On demand